

Analysis of Algorithms

Assignment 04

Name: Jose Ignacio Cabascango Fernandez

Link GitHub: <https://github.com/CodeWithJose1/Analysis-of-Algorithms/tree/main/Assignment-04/src>

1. Solve the 0/1 Knapsack problem using exhaustive search. Generate the problem, capacity, weights, and values randomly for a problem size n . You could also use a benchmark dataset if you wish. Find which is the biggest problem size n that you can solve in 30s of execution. You will have to record in a table the execution time for each $n = 3, 4, 5, 6, \dots$

The Knapsack Problem consists of a set of n items. Each item has:

- A weight w_i .
- A value v_i .

The knapsack has a maximum capacity W .

The goal is to maximize the total value of the selected items while keeping the total weight within the capacity. Exhaustive search solves the problem by generating and evaluating all possible candidate solutions. Therefore, with n items, the total number of possible subsets is:

$$2^n$$

and the time complexity is:

$$O(2^n)$$

```
1 /*
2  * Analysis of Algorithms - Assignment 04
3  *
4  * Name: Jose Ignacio Cabascango Fernandez
5  *
6  * 1. Solve the 0/1 Knapsack problem using exhaustive search. Generate the problem,
7  * capacity, weights, and values randomly for a problem size n. You could also use a
8  * benchmark dataset if you wish. Find which is the biggest problem size n that you
9  * can solve in 30s of execution. You will have to record in a table the execution
10 * time for each n = 3, 4, 5, 6, ...
11 */
12
13 #include <iostream>
14 #include <vector>
15 #include <random>
16 #include <chrono>
17 #include <iomanip>
18 using namespace std;
19
20 struct object{
21     int n;
22     int combinations;
23     double time;
24 };
25
26 int main() {
27     random_device rd;
28     mt19937 gen(rd());
29     uniform_int_distribution<int> dis(1, 1000);
30     vector<object> results;
31     for (int n = 3; n <= 28; n++) {
32         int combinations = 1;
33         double time = 0.0;
34         for (int i = 0; i < combinations; i++) {
35             // Generate random weights and values
36             vector<int> weights(n);
37             vector<int> values(n);
38             for (int j = 0; j < n; j++) {
39                 weights[j] = dis(gen);
40                 values[j] = dis(gen);
41             }
42             // Exhaustive search
43             int max_value = 0;
44             for (int mask = 0; mask < (1 << n); mask++) {
45                 int current_weight = 0;
46                 int current_value = 0;
47                 for (int j = 0; j < n; j++) {
48                     if (mask & (1 << j)) {
49                         current_weight += weights[j];
50                         current_value += values[j];
51                     }
52                 }
53                 if (current_weight <= 10000) {
54                     max_value = max(max_value, current_value);
55                 }
56             }
57             time += chrono::duration_cast<chrono::microseconds>(chrono::high_resolution_clock::now() - chrono::high_resolution_clock::now());
58         }
59         results.push_back({n, combinations, time});
60         combinations *= 2;
61     }
62     // Print results
63     cout << "n\tCombinations\tTime (seconds)\n";
64     for (const auto& result : results) {
65         cout << result.n << "\t" << result.combinations << "\t" << result.time / 1000000.0 << "\n";
66     }
67     return 0;
68 }
```

n	Combinations	Time (seconds)
3	8	0.000001
4	16	0.000001
5	32	0.000002
6	64	0.000005
7	128	0.000010
8	256	0.000021
9	512	0.000048
10	1024	0.000102
11	2048	0.000217
12	4096	0.000470
13	8192	0.000986
14	16384	0.002048
15	32768	0.004176
16	65536	0.008430
17	131072	0.017571
18	262144	0.036589
19	524288	0.077578
20	1048576	0.157498
21	2097152	0.328881
22	4194304	0.686600
23	8388608	1.386775
24	16777216	2.885984
25	33554432	5.942626
26	67108864	12.360392
27	134217728	25.352276
28	268435456	51.736111

30-seconds limit reached at n = 28
sh-5.35

The obtained results were:

n	Combinations	Time (s)	n	Combinations	Time (s)
---	--------------	----------	---	--------------	----------

3	8	0.000001	16	65,536	0.008450
4	16	0.000001	17	131,072	0.017571
5	32	0.000002	18	262,144	0.036589
6	64	0.000005	19	524,288	0.077578
7	128	0.000010	20	1,048,576	0.157498
8	256	0.000021	21	2,097,152	0.328881
9	512	0.000048	22	4,194,304	0.680600
10	1,024	0.000102	23	8,388,608	1.388.775
11	2,048	0.000217	24	16,777,216	2.885.984
12	4,096	0.000470	25	33,554,432	5.942.626
13	8,192	0.000986	26	67,108,864	12.300.392
14	16,384	0.002048	27	134,217,728	25.352.276
15	32,768	0.004176	28	268,435,456	51.736.111

The results clearly show the exponential growth of the exhaustive-search algorithm. Every time n increases by one, the number of possible combinations approximately doubles.

This is reflected directly in the execution time:

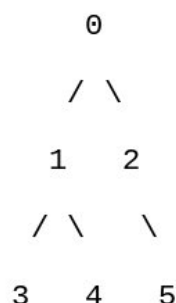
- For $n = 25$, the program required 5.94 s.
- For $n = 26$, it required 12.30 s.
- For $n = 27$, it required 25.35 s.
- For $n = 28$, it required 51.74 s.

The largest value of n that completed within the 30-second limit was $n = 27$, with an execution time of approximately 25.35 s, for $n = 28$, the execution time exceeded the limit, reaching approximately 51.74 s.

2. Implement a Depth First Search algorithm to print the values of each node in a graph. Decide which data structure to use: adjacency matrix or adjacency list.

An adjacency list is used to represent the graph. An adjacency list stores, for each vertex, the vertices directly connected to it. This representation is convenient for graph traversal, as the DFS algorithm only needs to visit existing edges.

Example graph:



Adjacency list:

```

0 → 1, 2
1 → 0, 3, 4
2 → 0, 5
3 → 1
4 → 1
5 → 2

```

DFS explores the graph by advancing as deeply as possible along each branch before backtracking.

The implementation uses:

- stack to manage the vertices to visit.

- visited to keep track of vertices that have already been visited.
- An adjacency list to represent the graph.

Using an adjacency list, DFS has a time complexity of:

$$\Theta(|V| + |E|)$$

where:

- V = number of vertices.
- E = number of edges.

Each vertex and edge is processed a constant number of times.

The screenshot shows a C++ IDE with a file named '2.DFS.cpp'. The code implements a Depth-First Search (DFS) algorithm on a graph. The graph is defined in the main function as an adjacency list with 6 vertices and 5 edges. The DFS function is recursive and prints the vertices in the order they are visited. The output in the terminal is 'DFS traversal: 0 1 3 4 2 5'.

```

15 void DFS(const vector<vector<int>>& graph, int start) {
23     while (!is.empty()) {
33         for (int i = graph[current].size() - 1; i >= 0; i--) {
36             if (!visited[neighbor]) {
39                 // ...
40             }
41         }
42     }
43 }
44
45 int main() {
46     vector<vector<int>> graph = {
47         {1, 2},
48         {0, 3, 4},
49         {0, 5},
50         {1},
51         {1},
52         {2},
53     };
54     cout << "DFS traversal: ";
55     DFS(graph, 0);
56     cout << endl;
57     return 0;
58 }

```

DFS traversal: 0 1 3 4 2 5

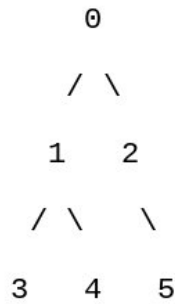
The result for the graph starting at vertex 0 is:

DFS traversal: 0 1 3 4 2 5

3. Implement a Breadth First algorithm to print the values of each node in a graph. Decide which data structure to use: adjacency matrix or adjacency list.

An adjacency list is also used for BFS. I used the same graph from Question 2:

Example graph:



Adjacency list:

0 → 1, 2
1 → 0, 3, 4
2 → 0, 5
3 → 1
4 → 1
5 → 2

BFS explores the graph level by level. It first visits the starting vertex, then all its neighbors, followed by the neighbors of those vertices.

The implementation uses:

- queue to manage the vertices using FIFO order.
- visited to avoid visiting the same vertex more than once.
- An adjacency list to represent the graph.

Using an adjacency list, BFS has a time complexity of:

$$\Theta(|V| + |E|)$$

where:

- V = number of vertices.
- E = number of edges.

```
15 void BFS(const vector<vector<int>>& graph, int start) {
21     visited[start] = true;
22     q.push(start);
23
24     while(!q.empty()) {
25         int current = q.front();
26         q.pop();
27
28         cout << current << " ";
29
30         for (int neighbor : graph[current]) {
31             if (!visited[neighbor]) {
32                 visited[neighbor] = true;
33                 q.push(neighbor);
34             }
35         }
36     }
37 }
38
39 int main() {
40     vector<vector<int>> graph = {
41         {1, 2},
42         {0, 3, 4},
43         {0, 5},
44         {1},
45         {1},
46         {2},
47     };
48
49     cout << "BFS traversal: ";
50     BFS(graph, 0);
51     cout << endl;
52
53     return 0;
54 }
```

BFS traversal: 0 1 2 3 4 5

Starting from vertex 0, the output is:

BFS traversal: 0 1 2 3 4 5

The algorithm first visits 0, then its neighbors 1 and 2, and finally the next level 3, 4, and 5.